Cascade Pipeline

Containerized VFX/Animation production pipeline — deployment, release engineering, and CI/CD

Role	DevOps / Pipeline Engineering
Domain	VFX & Animation Studio infrastructure
Stack	Docker · Docker Compose · PostgreSQL · Redis · Nginx · GitHub Actions · Python (uv) · Node/Yarn · FFmpeg · Bash/PowerShell
Repositories	`cascade-deploy`, `cascade-core`, `polaris-tools`

Legend:* *[OK] implemented ***[plan] designed / target-state*** *The diagrams that follow show the full intended architecture; the implementation-status table at the end is the source of truth for what is wired up today.*

---

1. Project overview

Cascade is the studio's self-hosted production-tracking and pipeline server for animation and VFX work. This project is the deployment of that server, plus the delivery pipeline that ships the studio's own customizations into it.

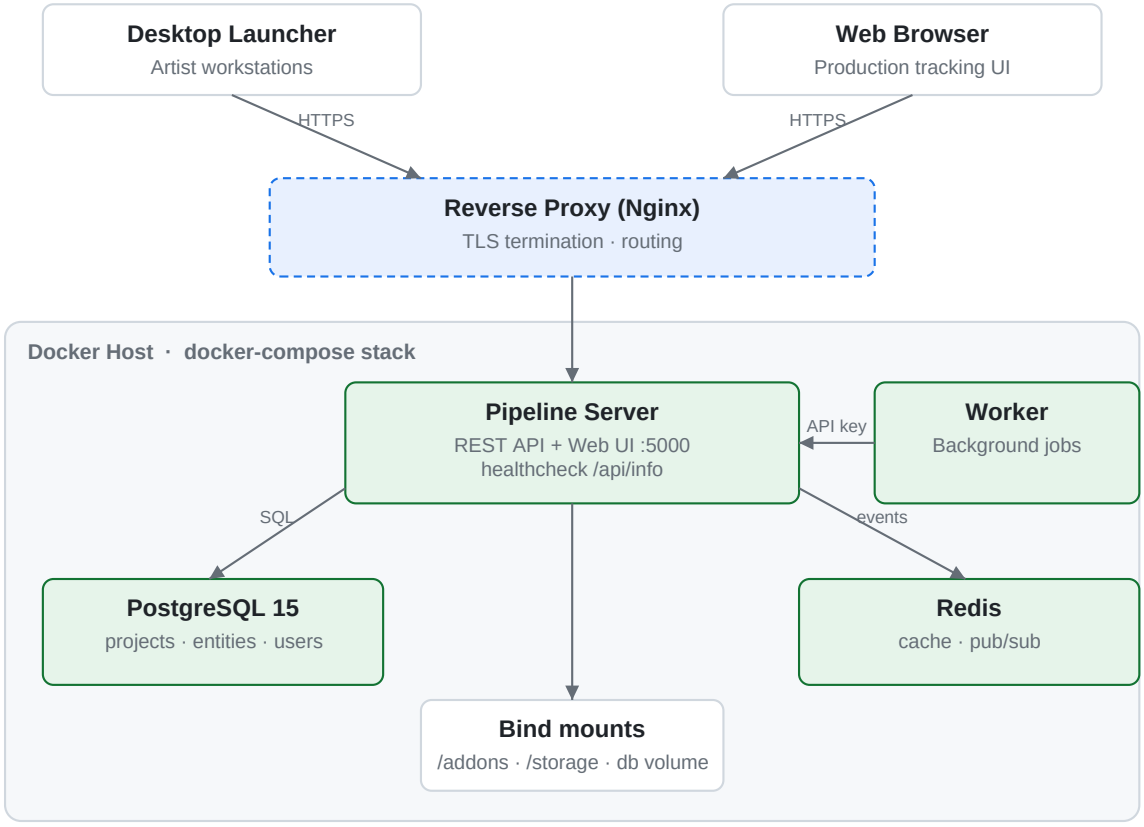
The deployment is composed of three repositories that fit together cleanly:

Repository	Role	Output
`cascade-deploy`	Deployment & orchestration — the Compose stack, custom server image, ops tooling	Running Cascade stack
`cascade-core`	Fork of the core pipeline addon (publish plugins, business logic)	`core-1.9.x.zip`
`polaris-tools`	Studio-specific addon — burnins & slates, replaces a vendor-licensed component	`polaris-tools-1.1.0.zip`

The two addon repos build versioned `.zip` packages; **cascade-deploy** is where they are mounted, served, and run. A change to a plugin therefore flows: *edit addon* → *CI lints & tests* → *build package* → *promote through environments* → `docker compose up` *in production.*

2. System architecture (runtime)

Clients reach the stack through a reverse proxy that terminates TLS and forwards to the Cascade server's REST API and web UI on port 5000. The server persists tracking data in PostgreSQL, uses Redis for caching and its real-time event stream, and loads the studio's custom addons from a bind-mounted directory. Published media lands under a dedicated storage mount. A worker service authenticates back to the server with an API key and runs background jobs.



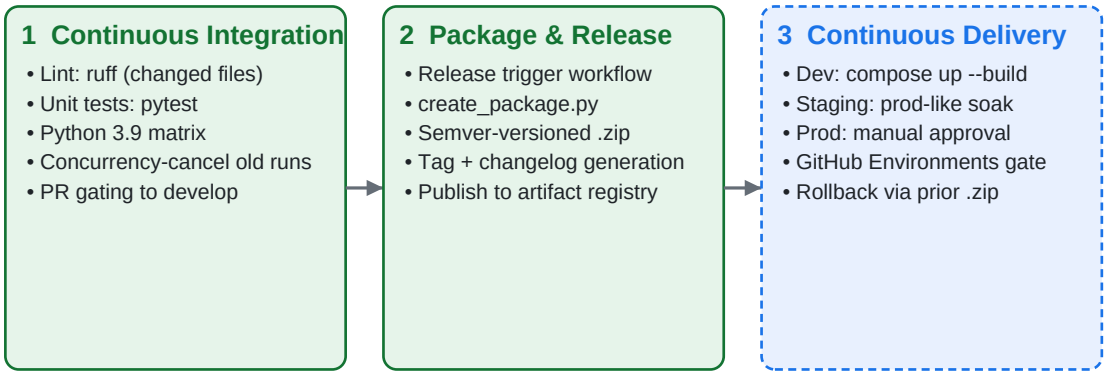
Component reference

Service	Image	Exposes	Healthcheck	Persistence
server	cascade-server:${TAG}	5000	curl /api/info	./addons, ./storage (bind)
postgres	postgres:15	5432 (internal)	pg_isready -U cascade	db named volume
redis	redis:alpine	6379 (internal)	redis-cli ping	ephemeral
worker	cascade-worker:latest	host network	waits for server healthy	docker.sock

Dependency and startup ordering is enforced with Compose health conditions: `postgres` must report `pg_isready` before `server` starts, and `server`'s `/api/info` probe must pass before the worker is brought up.

3. CI/CD pipeline

The pipeline's job is to guarantee that only linted, tested, versioned addon packages reach production, and that promotion between environments is deliberate and auditable.



Stage detail

Continuous Integration. Every pull request targeting `develop` runs ruff linting (scoped to changed files, versioned from `pyproject.toml`) and a pytest unit-test matrix on Python 3.9. A concurrency policy cancels superseded runs so only the latest commit on a PR is tested. Status checks must be green before merge.

Package & Release. A `workflow_dispatch` release trigger cuts a tag and changelog, `create_package.py` builds the versioned addon `.zip`, and an `upload_to_artifact_registry` job publishes it on `release: published`.

Continuous Delivery (planned). The released package is rolled through environments: development pulls automatically, **staging** runs a prod-like soak test, and **production** deploys only behind a manual approval gate using GitHub Environments protection rules. Rollback is a redeploy of the prior package version — no image rebuild needed.

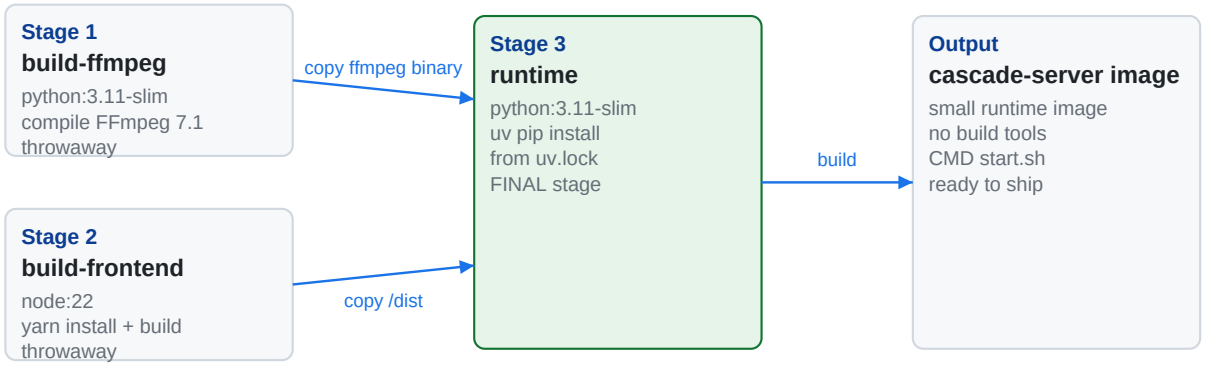
4. Environments & promotion

Three environments, each with strict expectations on what is allowed to differ between them. The same Compose file and same packaged addons run end-to-end; only data, secrets, and TLS configuration change between stages.

	Development	Staging	Production
Purpose	Build features fast	Validate the release candidate	Serve artists
Data	Demo / disposable	Prod-like, disposable	Real, backed up nightly
Addons	Live bind-mount of source	Built .zip (same as prod)	Built .zip
TLS	Off (localhost)	Self-signed / staging cert	Real cert at reverse proxy
Deploy trigger	compose up --build	Auto on merge to develop	Manual approval after staging
Secrets	Defaults / .env	Staging secrets	Vaulted production secrets

5. Server image build pipeline

The Cascade server image is produced by a multi-stage Dockerfile that keeps the runtime image small by compiling heavy tooling in throwaway stages.



Static FFmpeg — compiled once with GPL+GnuTLS and runtime CPU detection; only the resulting binaries are copied into the runtime stage. The build toolchain (gcc, make, headers) never reaches the final image.

Frontend — built with Yarn in a Node 22 stage; only the `dist/` artifact is copied forward.

Backend — Python dependencies installed with `uv` (mounted from `ghcr.io/astral-sh/uv`) for fast, reproducible installs from `uv.lock`.

6. Operations runbook

Cross-platform task runners (`Makefile` for Unix, `manage.ps1` for Windows) expose the same verbs so operators on either OS use identical commands.

Task	Unix	Windows
Bring stack up	`docker compose up -d`	`docker compose up -d`
Apply settings template	`make setup`	`./manage.ps1 setup`
Create demo projects	`make demo`	`./manage.ps1 demo`
Open a DB shell	`make dbshell`	`./manage.ps1 dbshell`
Hot-reload server	`make reload`	`./manage.ps1 reload`
Pull + rebuild server	`make update`	`./manage.ps1 update`
Build server image	`make build`	`./manage.ps1 build`
Backup one project	`make dump project=<p>`	`./manage.ps1 dump <p>`
Restore one project	`make restore project=<p>`	`./manage.ps1 restore <p>`

Bootstrapping: `settings/template.json` seeds the initial admin user and a service account API key on first `setup`.

Backups: `scripts/backup_cascade.sh` runs a daily `pg_dump` of the database via cron (4 AM) into a dated `.sql` file. Per-project dump/restore targets handle granular project migration (schema + project rows + product types).

Scaling workers: `docker-compose.worker.yml` defines additional workers that attach to a healthy server with `CASCADE_API_KEY` / `CASCADE_SERVER_URL`, allowing processing capacity to scale horizontally.

7. Security & secrets

- **TLS termination at the reverse proxy.** Only the proxy is internet-facing; `postgres` and `redis` are never published — they are only `exposed` on the internal Compose network.
- **Per-environment secrets.** `POSTGRES_PASSWORD`, `CASCADE_API_KEY`, and admin credentials live in `.env` files locally and as GitHub Environment secrets in CI. In-repo defaults are clearly marked as development placeholders and are rotated before any internet-facing deployment.
- **Least privilege.** The worker mounts `docker.sock`, so it runs only on trusted hosts. Production isolates worker hosts from the control plane.
- **Auditability.** Every production change is a merged PR plus an approved GitHub Environment deployment — no manual edits on the host. Production deploys leave a complete audit trail.

8. Observability (target state)

The implemented Compose stack already publishes health probes (`/api/info` on the server, `pg_isready` on Postgres). The planned observability layer adds:

- Container and host metrics via cAdvisor + Prometheus; dashboards in Grafana.
- Centralized logs (Docker logs → Loki) with health alerts on the existing server and Postgres probes.
- Backup success/failure alerting on the nightly pg_dump cron job.

9. Implementation status

Source of truth for what is live today vs. what is designed and queued for delivery.

Capability	Status	Location
Compose stack (server, postgres, redis, worker)	**[OK] Implemented**	`docker-compose.yml`
Health-gated startup ordering	**[OK] Implemented**	`docker-compose.yml`
Multi-stage server image (ffmpeg/frontend/backend)	**[OK] Implemented**	`Dockerfile`
Custom addons via bind-mount	**[OK] Implemented**	`addons/, cascade-core, polaris-tools`
Cross-platform ops tooling	**[OK] Implemented**	`Makefile, manage.ps1`
Nightly DB backup	**[OK] Implemented**	`scripts/backup_cascade.sh`
Horizontal worker scaling	**[OK] Implemented**	`docker-compose.worker.yml`
CI: ruff lint + pytest on PRs	**[OK] Implemented**	`cascade-core .github/workflows`
Release tagging + artifact publish	**[OK] Implemented**	`release_trigger, upload_to_registry`
Reverse proxy + TLS	**[plan] Designed**	`target`
Dev → staging → prod promotion w/ approval gates	**[plan] Designed**	`target`
Secrets via env/vault per environment	**[plan] Designed**	`target`
Metrics / logs / alerting	**[plan] Designed**	`target`

10. DevOps skills demonstrated

Containerization & orchestration. Multi-service Docker Compose with health-gated dependencies, named volumes vs. bind-mounts, and horizontal worker scaling. Compose-based dependency ordering ensures the database is ready before the API starts, and the API is healthy before workers attach to it.

Image optimization. Multi-stage Docker builds that compile FFmpeg and the frontend in disposable stages, keeping the runtime image lean. Reproducible Python installs with `uv` and a lockfile.

CI/CD. GitHub Actions for parallel lint/test gates, concurrency control, artifact-based releases, and a designed dev → staging → prod promotion model with approval gates. PRs cannot merge without passing checks; production cannot deploy without explicit human approval.

Release engineering. Semver-versioned, independently deployable addon packages decoupled from the server image. Rollback is a redeploy of the previous package version — no image rebuild required, no DB migration to undo.

Cross-platform automation. Parity between `Makefile` and `manage.ps1` so operators on Linux/macOS and Windows use identical task verbs. Reduces operator error and training overhead.

Operability. Bootstrap, hot-reload, scheduled backups, and granular per-project dump/restore procedures. Designed so an on-call engineer can recover or migrate a single project without touching anything else.

Security posture. Internal-only data services (Postgres & Redis never publish to host), TLS at the edge, and per-environment secret management. Credentials never live in committed files outside of clearly-marked development placeholders.

Next on the roadmap

- Wire up the reverse proxy (Nginx) with Let's Encrypt for TLS termination.
- Land the GitHub Environments protection rules so prod requires a named approver — closes the manual-deploy gap.
- Move secrets out of `.env` files into a vault (Doppler / 1Password Connect / HashiCorp Vault — under evaluation).
- Stand up Prometheus + Grafana with dashboards driven by the Compose healthchecks already defined.

Architecture documentation for the Cascade Pipeline. Diagrams generated programmatically for this document.